BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

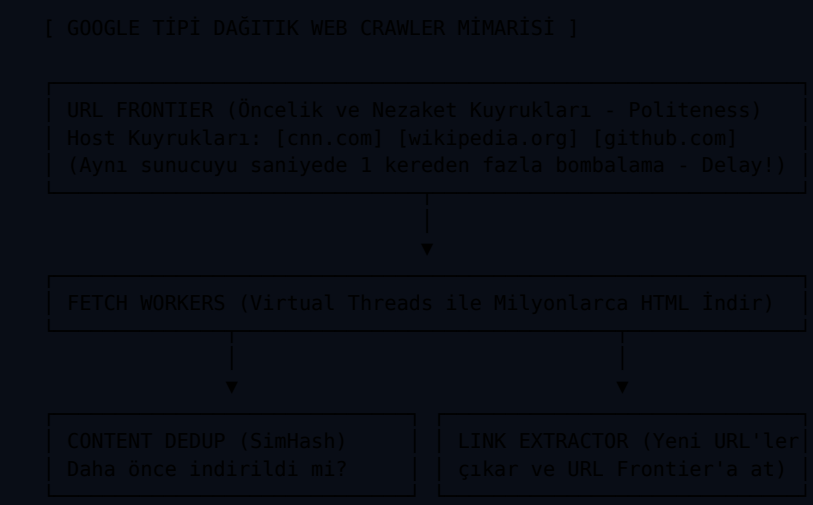
40

Web Gezgini (Web Crawler) Tasarımı: URL Frontier, robots.txt ve Duplication

Designing a Web Crawler: URL Frontier, Politeness & Deduplication

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki tüm kitapları tek tek okuyup fihrist çıkaran çalışkan bir robot düşünün (Web Gezgini / Googlebot). Robot bir sayfa okur, sayfanın altındaki dipnotlarda başka kitapların referansını görür (Link Extraction). O yeni kitapları 'Okunacaklar Listesi'ne (URL Frontier) ekler. Ancak robot çok terbiyelidir (Politeness): Aynı yazarın odasına saniyede 100 kez girip kapıyı çalmaz; her yazarın arasına 2 saniye bekleme süresi koyar. Ayrıca daha önce okuduğu bir kitabı tekrar okumamak için hafızasındaki parmak izlerini (Deduplication) kontrol eder!

🇬🇧 ENGLISH (ELI5)

Imagine an automated librarian cataloging every book ever printed (Googlebot). It opens a book, reads the citations at the bottom (Link Extraction), and writes those new book titles onto a massive 'To-Read Clipboard' (URL Frontier). The librarian is strictly disciplined and polite (Politeness): it never pesters the same author 100 times per second, inserting a courteous 1-second delay between visits. It also checks a fingerprint catalog (Deduplication) so it never rereads identical duplicate books!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Web Crawler Mimarisi: 1) URL Frontier: İndirilecek URL'leri tutan kalıcı kuyruk sistemi. İki temel gereksinim: A) Önceliklendirme (Priority): Kaliteli ve popüler sayfalar önce taranır (PageRank tabanlı). B) Nezaket (Politeness): Aynı hedef alan adına (Host/Domain) giden isteklerin arasına gecikme (ör. 1000ms) konur; hedef sunucu DDoS saldırısına uğramaz. 2) DNS Önbellekleme: Milyonlarca istekte DNS sorguları darboğaz olmasın diye yerel DNS Cache tutulur. 3) İçerik Tekilleştirme (Content Deduplication): İnternetteki içeriğin %30'u mükerrerdir. HTML içeriğinin SimHash veya MinHash parmak izi alınarak Hamming Mesafesi (Hamming Distance) ile benzerlik kontrol edilir. 4) robots.txt: Taranmadan önce sitenin `robots.txt` kuralları okunur ve önbelleğe alınır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architecture of a Distributed Web Crawler (Googlebot scale): 1) The URL Frontier: Governs URL scheduling via a dual-tiered queueing engine: A) Priority Queues order pages by freshness and PageRank. B) Politeness Queues group URLs by domain hostname, ensuring worker threads enforce delay intervals (e.g., 500ms) to avoid crashing target web hosts. 2) Dedicated Local DNS Caching: DNS lookups represent 30%+ of network latency; crawlers maintain in-memory high-throughput DNS caches. 3) Near-Duplicate Content Elimination: Over 30% of web pages are identical mirrors. Crawlers compute cryptographic 64-bit `SimHash` or `MinHash` locality-sensitive signatures, comparing Hamming Distance to discard duplicates before indexing. 4) robots.txt Compliance: Robots Exclusion Protocol files are fetched and cached in memory prior to scraping target paths.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: SimHash Benzerlik ve Tekilleştirme (Deduplication) Algoritması
import java.math.BigInteger;

public class SimHashDeduplicator {
    public static long computeSimHash(String text) {
        int[] v = new int[64];
        String[] words = text.toLowerCase().split("\\s+");
        for (String word : words) {
            long hash = word.hashCode(); // Basit hash simülasyonu
            for (int i = 0; i < 64; i++) {
                if (((hash >> i) & 1L) == 1L) v[i]++;
                else v[i]--;
            }
        }
        long simHash = 0L;
        for (int i = 0; i < 64; i++) {
            if (v[i] > 0) simHash |= (1L << i);
        }
        return simHash;
    }

    // İki parmak izi arasındaki Hamming Mesafesi (Farklı bit sayısı)
    public static int hammingDistance(long h1, long h2) {
        return Long.bitCount(h1 ^ h2);
    }
}

void main() {
    String doc1 = "Java 25 and Spring Boot system design interview guide";
    String doc2 = "Java 25 and Spring Boot system design interview tutorial"; // Çok benzer!

    long hash1 = SimHashDeduplicator.computeSimHash(doc1);
    long hash2 = SimHashDeduplicator.computeSimHash(doc2);

    int distance = SimHashDeduplicator.hammingDistance(hash1, hash2);
    IO.println("Hamming Mesafesi: " + distance + " bit (Mesafe <= 3 ise MÜKERRER KABUL EDİLİR!)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'BFS (Breadth-First Search) ile tararım' deyip geçmeyin! Saf BFS yaparsanız tek bir sitedeki 100.000 linki sırayla indirip o web sitesini çökertebilirsiniz (İhlal: Impolite). Nezaket (Politeness) kuyrukları ile farklı hostların dönüşümlü taranması zorunludur.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose naive Breadth-First Search (BFS) for web crawling. A pure BFS traversal hammers a single domain with thousands of concurrent connections, effectively staging a DDoS attack. Explicitly design Politeness queues.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• URL Frontier, önceliklendirme ve nezaket (politeness) dengesini kuran omurgadır.

• SimHash ve MinHash algoritmaları milyarlarca web sayfasındaki benzerlikleri saniyede eler.

• Yerel DNS önbellekleme, web tarama hızını %30 artıran en kritik optimizasyondur.

🎯 Key Takeaways for the Interview

• The URL Frontier enforces both crawl prioritization and domain politeness constraints.

• SimHash algorithms detect near-duplicate pages in constant time using Hamming distance.

• Dedicated in-memory DNS caching eliminates substantial latency bottlenecks during massive crawls.